

RESTful SPFI (2.0.0)

Download OpenAPI specification: [Download](#)

A RESTful Service Provider Fulfillment API.

In principal, this interface connects three types of entities, such that they can trigger actions and exchange information:

1. *Application Providers*:

An Application Provider implements the SPFI and therefore functions as the "server part" of the SPFI interface.

They provide a Tenant-aware application and offer actions on and between those Tenants that are triggered by an *Application Manager*. Additionally, they allow *Application Managers* to fetch status information of the managed Tenants.

2. *Application Manager*:

An Application Manager is the "client part" of the SPFI.

It interacts with *Application Providers* to trigger desired actions and queries for the results of those actions on the resources that are offered by the SPFI. It has access to the *Application Consumer's* contractual situation for contracts that managed via SAP mechanism on-behalf of the *Application Providers*. As a result, it acts as an intermediary between *Application Providers* and *Application Consumers* to achieve the known contractual state.

3. *Application Consumers*:

The Application Consumers are the customers of the *Application Providers* and consume their services.

Tenants

Tenants are the central entity of the SPFI:

An *Application Provider* is supposed to enable the provisioning and management of Tenants for the application it provides. This allows the trusted *Application Managers* to trigger the provisioning of Tenants, retrieve the current status of their provisioned Tenants, as well as to modify / delete them. To enable *Application Providers* to communicate the current state of a Tenant to the *Application Manager*, a Tenant has 4 defined regular lifecycle phases - namely "activation", "updating", "blocking", and "deletion" - and 3 error states, namely "in self-recoverable error", "in recoverable error", and "final error".

The Tenant lifecycle states and their potential transitions are depicted in Figure 1 below and explained in more detail in the following paragraphs:

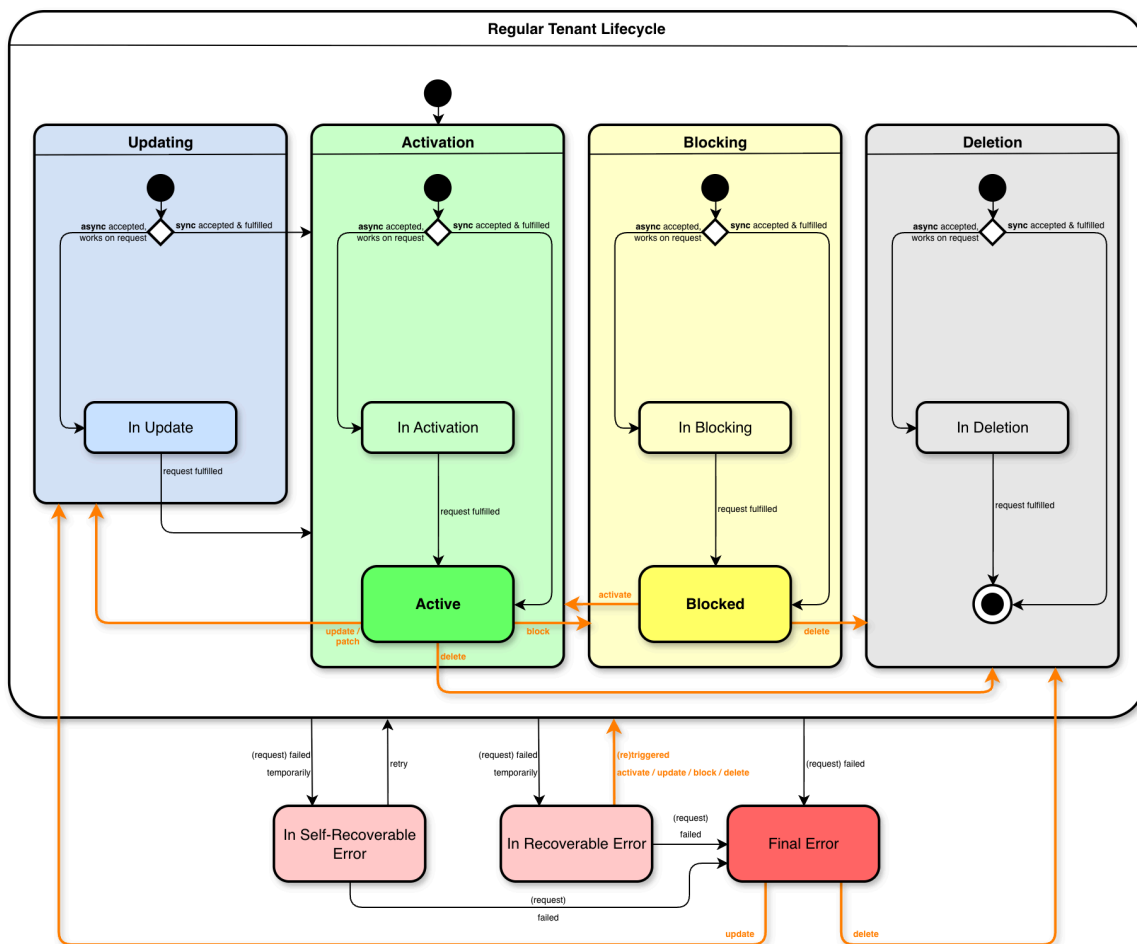


Figure 1: Tenant Lifecycle States

Each regular lifecycle phase allows an *Application Provider* to..

1. ..accept and immediately act on the fulfillment of the request with a "In *" state or
2. ..accept and immediately fulfill the request synchronously with the desired (temporary final) state - namely "active", "blocked", or a gone resource.

The lifecycle of a Tenant starts with the "activation" phase and ends with its "deletion". From each state a Tenant can transition into an error, if some operation or the Tenant itself fails. Here, the protocol distinguishes between recoverable / temporary errors and non-recoverable / final errors:

- **In Self-Recoverable Error:**

The "In Self-Recoverable Error" state indicates to the *Application Manager* that the *Application Provider* will resolve the error situation, identify that the error is resolved and recover itself without the need for any external intervention by the *Application Manager*.

- **In Recoverable Error:**

The "In Recoverable Error" state indicates to the *Application Manager* that the *Application Provider* will not identify when the error is resolved, but requires an external trigger by the *Application Manager* to retry the operation.

- **Final Error:**

The "Final Error" state indicates to the *Application Manager* that the error is not recoverable and the error must be resolved by manual intervention.

In general, there are two types of lifecycle state transitions:

1. **Provider-internal transitions:**

Lifecycle state transitions that are performed internally by the *Application Provider* are depicted as black arrows.

2. **Provider-external-triggered transitions:**

Lifecycle state transitions that are triggered externally by the *Application Manager* to be performed

by the *Application Provider* are depicted as orange arrows.

The Provider-internal transitions depend on the way an *Application Provider* can create/update/delete its Tenant resources - synchronously or asynchronously - as well as how/if it can recover from occurring errors. This can be different for each lifecycle phase or error situation.

In contrast to that, Provider-external-triggered transitions are fixed by the protocol:

1. Transition to "deletion" phase:

A Tenant can be externally requested to be transitioned to the "deletion" phase by use of the REST verb `DELETE` on the Tenant resource, if its state is "active", "blocked", or in a recoverable / final error.

2. Transition to "blocking" phase:

A Tenant can be externally requested to be transitioned to the "blocking" phase by use of the REST verb `POST` with `{ "state": "blocked" }` on the Tenant resource's `/status` endpoint, if it is in "active" state.

3. Transition to "activation" phase:

A Tenant can be externally requested to be transitioned to the "activation" phase by use of the REST verb `POST` with `{ "state": "active" }` on the Tenant resource's `/status` endpoint, if it is in "blocked" state.

4. Transition to "updating" phase:

A Tenant can be externally requested to be transitioned to the "updating" phase by use of the REST verb `PUT` on the Tenant resource, if it is in "active" state or in a recoverable / final error.

5. Transition from error states:

If a Tenant is in an error state, an external-triggered transition is only possible for the "In Recoverable Error" and "Final Error" states.

6. Impossible Transitions:

An external-triggered transition is not possible while a state change is actively performed, i.e. while the Tenant is in an "In *" state.

Get description of (many) Tenants

Get the description of multiple / all Tenants.

This information includes the definition of the Tenant, as well as its current provisioning status.

During initial provisioning or other actions performed on a Tenant, it can be beneficial for an *Application Manager* to get information about status changes timely. The *Application Manager* is advised to query individual resources by relying on the `GET tenants/{id}` endpoint and the received `Retry-After` header on a `'200' OK` response (if available) to establish an optimal polling interval. The *Application Provider* must only show the *Application Consumers'* Tenants that are associated to the querying *Application Manager* based on the confirmed identity of the *Manager* defined by Mutual TLS.

Important Note:

The functionality offered by this request is purely **optional** functionality, such that *Application Providers* are free to not support it. This might be the case, if an *Application Provider* fears that it cannot handle the potential load, if e.g. a streaming approach is not supported yet.

In this case, the *Application Provider* should return a `'501' Not Implemented` to indicate this limitation to the *Application Manager*.

HEADER PARAMETERS

Select	Array of strings (TenantResourceId) [items <code>^[a-zA-Z0-9-._~]*\$</code>] Allows to specify a subset of resources in order to limit the result set to the information of the specified set of resources. Can be used to significantly limit the information that is sent over the wire. Defaults to every resource. Important Note: The functionality offered by this header is purely optional functionality, such that <i>Application Providers</i> are free to not support it. If the <i>Application Provider</i> does not support this header and its functionality, this header will be ignored and the set of all resources will be returned instead.
--------	---

Responses

✓ **200 OK.** Returns the current information of the Tenants.

If any of the Tenants' status is non-final, a `Retry-After` header is sent with the response.

Hereby, the *Application Provider* tries to provide a best guess for the *Application Manager* on when to ask for an updated status.

RESPONSE HEADERS

Retry-After	string <integer> Example: <code>"120"</code> Indicates how many seconds the <i>Application Manager</i> should wait after it received this response, before making a follow-up request, when polling for the resource's state.
Content-Encoding >	gzip (string) Lists the applied encoding(s), such that the <i>Application Manager</i> is enabled to decode the message body.
Cache-Control	string Value: <code>"no-store"</code> Instructs all nodes on the the way of the response on how to treat the response data. In case a Server Sent Events stream is used, this value should be set to <code>no-store</code>.

RESPONSE SCHEMA: application/json

Array [	
id	string (TenantResourceId) <code>^[a-zA-Z0-9-._~]*\$</code> The unique identifier of the Tenant resource.

The *Application Provider* as the owning party of this identifier must ensure that this identifier is safe to be used within a URI.

This means the identifier must not include reserved or unsafe characters in the context of URIs as defined in [IETF RFC 3986 \(Section 2.3\)](#). Hence, only letters, digits, `-`, `.`, `_`, and `~` are allowed.

While not obligatory, it is strongly recommended to use the provided `sapId`, i.e. the SAP-specific resource identifier as the resource identifier.

`application >`

object (Application)

Application properties that are solely provided by the *Application Provider* as part of or as a result of the provisioning process.

These properties are not part of the desired state specified by the *Application Manager* and hence can only be modified by the *Application Provider*.

`sapId`

string (SAPId) `^[1-9][0-9]*$`

The SAP-specific resource identifier.

This identifier can be used by the *Application Provider* to validate, if a matching SAP purchase exists for this request and associate it with its provisioned resource.

`customer >`

object (Customer)

The individual customer.

`contract >`

object (Contract)

The contractual details that are associated with the Tenant.

`products >`

Array of Product ID only (object) or Product ID & Quota (object) (Products) `non-empty`

The list of products that are associated to the Tenant.

`businessType`

string (BusinessType)

The type of Tenant from a business purpose perspective. For example, this information can be used to distinguish between Tenants that are used for development, test, or production. Alternatively, this attribute can indicate a particular service plan of the Tenant, like "demo", "free", or "premium".

`operationalType`

string (OperationalType)

A specific operation performed on the Tenant by the *Application Provider*.

`location >`

object (Location) `non-empty`

The logical and/or physical location of the Tenant.

An *Application Provider* is free to use only the set of attributes that are sufficient to identify the desired Tenant location. Each attribute is semantically well-defined to allow a potential use of this information by the *Application Manager*.

`initialUsers` >

Array of objects (InitialUsers) non-empty

The initial users that should be granted immediate access to the Tenant.

`hostTenantSpecification` >

object (HostTenantSpecification)

A wrapper for host tenant specific properties that should be mixed into the regular Tenant specification.

Includes *Application Provider* specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular *Application Provider*.

`mixinSpecifications` >

Array of objects (MixinSpecifications) non-empty

A wrapper for generic properties that should be mixed into the regular Tenant specification.

Includes *Application Provider* specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular *Application Provider*.

`additionalProperties` >

object (AdditionalPropertiesFromManager) [1 .. 100] properties

Freely definable properties, which are provided by the *Application Manager*.

Those properties are not generally applicable, but only relevant for a specific *Application Provider* in order to successfully provision/manage a Tenant dependent on its specific configuration needs.

Those properties will simply be routed through to the *Application Provider*, but won't be interpreted or otherwise acted on by the *Application Manager*, because they are not well-defined on the SPFI protocol level.

`status` >

object (LifecycleStatusWithUpdateAndBlock)

The lifecycle status of the Tenant.

]

> 400 Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

> 401 Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

> 429 Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

> 500 Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

> 501 Not Implemented.

The *Application Manager's* request is valid, but the *Application Provider* choose to not implement the optional functionality of the protocol at this point in time.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

GET /tenants

Response samples

200**400****401****429****500****501****503**

Content type

application/json

Copy Expand all Collapse all

```
[
  - {
    "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24",
    + "application": { ... },
    "sapId": "324928342811231213",
    + "customer": { ... },
    + "contract": { ... },
    + "products": [ ... ],
    "businessType": "ZH421",
    "operationalType": "string",
    + "location": { ... },
    + "initialUsers": [ ... ],
    + "hostTenantSpecification": { ... },
    + "mixinSpecifications": [ ... ],
    + "additionalProperties": { ... },
    + "status": { ... }
  }
]
```

Trigger provisioning of a Tenant

Triggers the provisioning of a new Tenant as defined in the payload of the request.

Whenever the *Application Provider* receives such a request, it must validate that there is not already a Tenant for the given `sapId`. Only if a Tenant does not already exist with such an identifier, the Tenant will be provisioned. Otherwise, a `'409' Conflict` must be returned, which must indicate the conflicting `id` – that already exists for the given `sapId` – as part of the resource's address in the `Location` header and in the error response body's `target` attribute.

This situation can mainly occur, whenever..

- .. the `'201' Created` or `'202' Accepted` response of the *Application Provider* times out before it reaches the *Application Manager* and the *Application Manager* never gets hold of the resource `id`.
- .. the *Application Manager* lost possession of the *Application Provider's* resource identifier and wants to actively re-retrieve it via the known `sapId`.

In all cases, as a result of those checks a provisioning request becomes idempotent which avoids an "overprovisioning".

If the request is successful, the *Application Provider* should respond with a different status code dependent on the kind of provisioning:

- **a) Fast Provisioning**

The *Application Provider* should respond with `'201' Created`, if the Tenant **can** be provisioned during the HTTP request's lifetime, i.e. before the request times out. For example, a fast provisioning could be achieved, if the Tenant provisioning process requires only one or multiple additional entries in a database.

- **b) Slow Provisioning**

The *Application Provider* should respond early with `'202' Accepted`, if the Tenant **cannot** be provisioned during the HTTP request's lifetime, to avoid a timeout of the request. For example, a slow provisioning has to be assumed, if the Tenant provisioning requires a long-running system provisioning procedure or even manual intervention.

If unsure, a "Slow Provisioning" should be assumed by the *Application Provider*.
A successful response will return the *Application Provider's* identifier of the (to-be) provisioned Tenant in the payload as well as in the `Location` header.
Hereby, the *Application Provider* has the option to return a "temporary resource identifier" instead of the regular permanent resource identifier. This enables the *Application Provider* to return a response quickly, in case the permanent identifier is not yet available or not yet known to the *Application Provider*. Similarly, this identifier can point to an internal job resource that is performing the triggered action and used as a "proxy identifier" for progress tracking by the *Application Provider*.
Once / In case a permanent identifier is available, the *Application Provider* can indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and informs the *Application Manager* about this new location accordingly.
From the perspective of the *Application Manager*, it can in all cases simply track the status of the resource by the provided `Location` header URI, such that the internal workings of the *Application Provider's* provisioning process are almost fully abstracted.

AUTHORIZATIONS: > *Mutual TLS*

REQUEST BODY SCHEMA: application/json
required

The desired Tenant that should be created as a result of this request and the customer details that this Tenant is associated with.

<code>sapId</code> <i>required</i>	string (SAPId) <code>^[1-9][0-9]*\$</code> The SAP-specific resource identifier. This identifier can be used by the <i>Application Provider</i> to validate, if a matching SAP purchase exists for this request and associate it with its provisioned resource.
<code>customer</code> > <i>required</i>	object (Customer) The individual customer.
<code>contract</code> >	object (Contract) The contractual details that are associated with the Tenant.
<code>products</code> >	Array of Product ID only (object) or Product ID & Quota (object) (Products) <code>non-empty</code> The list of products that are associated to the Tenant.

<code>businessType</code>	string (BusinessType) The type of Tenant from a business purpose perspective. For example, this information can be used to distinguish between Tenants that are used for development, test, or production. Alternatively, this attribute can indicate a particular service plan of the Tenant, like "demo", "free", or "premium".
<code>operationalType</code>	string (OperationalType) A specific operation performed on the Tenant by the <i>Application Provider</i>.
<code>location</code> > <code>required</code>	object (Location) <code>non-empty</code> The logical and/or physical location of the Tenant. An <i>Application Provider</i> is free to use only the set of attributes that are sufficient to identify the desired Tenant location. Each attribute is semantically well-defined to allow a potential use of this information by the <i>Application Manager</i>.
<code>initialUsers</code> >	Array of objects (InitialUsers) <code>non-empty</code> The initial users that should be granted immediate access to the Tenant.
<code>hostTenantSpecification</code> >	object (HostTenantSpecification) A wrapper for host tenant specific properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.
<code>mixinSpecifications</code> >	Array of objects (MixinSpecifications) <code>non-empty</code> A wrapper for generic properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.

`additionalProperties` >

object (AdditionalPropertiesFromManager) `[ 1 .. 100 ] properties`

Freely definable properties, which are provided by the *Application Manager*.

Those properties are not generally applicable, but only relevant for a specific *Application Provider* in order to successfully provision/manage a Tenant dependent on its specific configuration needs.

Those properties will simply be routed through to the *Application Provider*, but won't be interpreted or otherwise acted on by the *Application Manager*, because they are not well-defined on the SPFI protocol level.

Responses

> **201 Created.** Returns the *Application Provider's* identifier (`id`) of the Tenant.

The Tenant was synchronously provisioned. The `Location` header can be checked for further details on the created resource's status.

> **202 Accepted.** Returns the *Application Provider's* (temporary) identifier (`id`) of the Tenant.

The Tenant provisioning is performed asynchronously. As a result, the creation of this Tenant was accepted, but the Tenant is not yet created. It is instead still within its provisioning process. Therefore, the `Location` header should be checked, to observe the current status of the newly created resource.

> **400 Bad Request.**

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

> **401 Unauthorized.**

The *Application Manager* did not provide credentials or the credentials were invalid.

> **403 Forbidden.**

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

> **409 Conflict.**

The *Application Manager* wanted to create a Tenant resource that already exists at the *Application Provider*.

> **429 Too Many requests.**

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

> 500 Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

POST /tenants

Request samples

Payload**Content type**

application/json

Copy

Expand all

Collapse all

```
{
  "sapId": "324928342811231213",
  - "customer": {
    "id": "324928342811231213",
    "erpId": "0001211321",
    "name": "string",
    + "contact": { ... }
  },
  - "contract": {
    "validFrom": "2019-08-24",
    "validTo": "2019-08-24"
  },
  - "products": [
    + { ... }
  ],
  "businessType": "ZH421",
  "operationalType": "string",
  - "location": {
    + "geography": { ... },
    + "infrastructure": { ... }
  },
}
```

```
- "initialUsers": [
  + { ... }
],
- "hostTenantSpecification": {
  "group": "my-group",
  "version": "my-version",
  "type": "my-type",
  "spec": "{ \"p1\": \"v1\", \"p2\": \"v2\" }"
},
- "mixinSpecifications": [
  + { ... }
],
- "additionalProperties": {
  "managerCustomProperty": "manager-custom-value"
}
}
```

Response samples

201

202

400

401

403

409

429

500

503

Content type

application/json

Copy

```
{
  "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24"
}
```

Get description of a Tenant

Get the description of a single Tenant.

This information includes the definition of the Tenant, as well as its current provisioning status.

During initial provisioning or other actions performed on a Tenant, it can be beneficial for an *Application Manager* to get information about status changes timely. The *Application Manager* is advised to rely on the received `Retry-After` header on a `'200' OK` response (if available) to establish an optimal polling interval.

The *Application Provider* must only grant access to a Tenant's information, if it is associated to the querying *Application Manager* based on the confirmed identity of the *Manager* defined by Mutual TLS. In case the request was sent towards a moved Tenant resource identifier, the *Application Provider* must indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and informs the *Application Manager* about this new location accordingly.

AUTHORIZATIONS: >

Mutual TLS

PATH PARAMETERS

id

required

string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`Examples: `b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24``888133312123288888`The identifier of the Tenant resource.

Responses

✓ **200 OK.** Returns the current information of the Tenant.

If the Tenant's status is non-final, a `Retry-After` header is sent with the response.

Hereby, the *Application Provider* tries to provide a best guess for the *Application Manager* on when to ask for an updated status.

RESPONSE HEADERS

Retry-After

string <integer>

Example: `"120"`

Indicates how many seconds the *Application Manager* should wait after it received this response, before making a follow-up request, when polling for the resource's state.

Content-Encoding >

gzip (string)

Lists the applied encoding(s), such that the *Application Manager* is enabled to decode the message body.

Cache-Control

string

Value: `"no-store"`

Instructs all nodes on the the way of the response on how to treat the response data.

In case a Server Sent Events stream is used, this value should be set to

`no-store`.RESPONSE SCHEMA: application/json

id

string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`

The unique identifier of the Tenant resource.

The *Application Provider* as the owning party of this identifier must ensure that this identifier is safe to be used within a URI. This means the identifier must not include reserved or unsafe characters in the context of URIs as defined in [IETF RFC 3986 \(Section 2.3\)](#). Hence, only letters, digits, `-`, `.`, `_`, and `~` are allowed.

While not obligatory, it is strongly recommended to use the provided `sapId`, i.e. the SAP-specific resource identifier as the resource identifier.

<code>application ></code>	object (Application) Application properties that are solely provided by the <i>Application Provider</i> as part of or as a result of the provisioning process. These properties are not part of the desired state specified by the <i>Application Manager</i> and hence can only be modified by the <i>Application Provider</i>.
<code>sapId</code>	string (SAPId) <code>^[1-9][0-9]*\$</code> The SAP-specific resource identifier. This identifier can be used by the <i>Application Provider</i> to validate, if a matching SAP purchase exists for this request and associate it with its provisioned resource.
<code>customer ></code>	object (Customer) The individual customer.
<code>contract ></code>	object (Contract) The contractual details that are associated with the Tenant.
<code>products ></code>	Array of Product ID only (object) or Product ID & Quota (object) (Products) <code>non-empty</code> The list of products that are associated to the Tenant.
<code>businessType</code>	string (BusinessType) The type of Tenant from a business purpose perspective. For example, this information can be used to distinguish between Tenants that are used for development, test, or production. Alternatively, this attribute can indicate a particular service plan of the Tenant, like "demo", "free", or "premium".
<code>operationalType</code>	string (OperationalType) A specific operation performed on the Tenant by the <i>Application Provider</i>.
<code>location ></code>	object (Location) <code>non-empty</code> The logical and/or physical location of the Tenant. An <i>Application Provider</i> is free to use only the set of attributes that are sufficient to identify the desired Tenant location. Each attribute is semantically well-defined to allow a potential use of this information by the <i>Application Manager</i>.
<code>initialUsers ></code>	Array of objects (InitialUsers) <code>non-empty</code> The initial users that should be granted immediate access to the Tenant.

hostTenantSpecification >	object (HostTenantSpecification) A wrapper for host tenant specific properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.
mixinsSpecifications >	Array of objects (MixinSpecifications) non-empty A wrapper for generic properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.
additionalProperties >	object (AdditionalPropertiesFromManager) [1 .. 100] properties Freely definable properties, which are provided by the <i>Application Manager</i>. Those properties are not generally applicable, but only relevant for a specific <i>Application Provider</i> in order to successfully provision/manage a Tenant dependent on its specific configuration needs. Those properties will simply be routed through to the <i>Application Provider</i>, but won't be interpret or otherwise acted on by the <i>Application Manager</i>, because they are not well-defined on the SPFI protocol level.
status ▾	object (LifecycleStatusWithUpdateAndBlock) The lifecycle status of the Tenant.
state > required	In Activation (string) or Active (string) or In Update (string) or In Blocking (string) or Blocked (string) or In Deletion (string) or In Self-Recoverable Error (string) or In Recoverable Error (string) or Final Error (string) (LifecycleStatesWithUpdateAndBlock)
details >	Details: Generic (object) or Details: Blocked by Request (object) or Details: Blocked by Missing Payment (object) (LifecycleStatesDetails)
lastModified required	string <date-time> (LifecycleStatesLastModified) The last time this state has been modified. The date and time must be specified in accordance with IETF RFC 3339 (Section 5.6).

➤ **400** Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

➤ **401** Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

➤ **403** Forbidden.

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

➤ **404** Not Found. The *Application Manager* provided credentials, but did request a resource that

- either does not exist
- or that exists, but the credentials are not sufficient to perform the desired action on the specified resource

In the latter case, the existence of the resource should be disguised from the *Application Manager* to provide additional security compared to a `'403' Forbidden`.

➤ **410** Gone.

The Tenant resource has been permanently moved by the *Application Provider* to the location specified by the `Location` header with a new resource identifier as specified in the response body.

The *Application Manager* should update its resource reference accordingly to the new location and use this location in subsequent requests, because the *Application Provider* can retire the redirect if it is certain that the new location is well known.

To enforce the explicit handling of this response, a `'410' Gone` is used in favor of a `'301' Moved Permanently`, because many HTTP clients follow `301` redirects transparently by default.

In that case, the *Application Manager* would handle only the response of the subsequent (forwarded) request explicitly, such that the resource relocation information within the `301` response would stay unnoticed.

➤ **429** Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

➤ **500** Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

➤ **501** Not Implemented.

The *Application Manager's* request is valid, but the *Application Provider* choose to not implement the optional functionality of the protocol at this point in time.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

GET /tenants/{id}

Response samples

200**400****401****403****404****410****429****500****501****503****Content type**

application/json

[Copy](#) [Expand all](#) [Collapse all](#)

```
{
  "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24",
  - "application": {
    "globalTenantId": "12099f8d-73ce-4175-b53c-8080410d7ef8",
    + "endpoints": [ ... ],
    + "additionalProperties": { ... }
  },
  "sapId": "324928342811231213",
  - "customer": {
    "id": "324928342811231213",
    "erpId": "0001211321",
    "name": "string",
    + "contact": { ... }
  },
  - "contract": {
    "validFrom": "2019-08-24",
    "validTo": "2019-08-24"
  },
  - "products": [
    + { ... }
  ],
  "businessType": "ZH421",
  "operationalType": "string",
```

```

- "location": {
  + "geography": { ... },
  + "infrastructure": { ... }
},
- "initialUsers": [
  + { ... }
],
- "hostTenantSpecification": {
  "group": "my-group",
  "version": "my-version",
  "type": "my-type",
  "spec": "{ \"p1\": \"v1\", \"p2\": \"v2\" }"
},
- "mixinSpecifications": [
  + { ... }
],
- "additionalProperties": {
  "managerCustomProperty": "manager-custom-value"
},
- "status": {
  "state": "in-activation",
  + "details": { ... },
  "lastModified": "2019-08-24T14:15:22Z"
}
}

```

Update properties of a Tenant

Update the properties of a single Tenant.

Only the mutable parts of a Tenant resource can be modified, but no lifecycle states.

To trigger an action that changes the Tenant's desired lifecycle state, the status endpoint `/status` or the `DELETE` verb must be used.

The *Application Provider* will check, if the Tenant is in a lifecycle state that allows its update - namely in state "active", "in-recoverable-error", or "final-error". If this check is not successful, the *Application Provider* should respond with a `'409' Conflict` indicating this conflict. If the request is successful, the *Application Provider* should respond with a different status code dependent on the kind of update process:

- **a) Fast Update**

The *Application Provider* should respond with `'200' OK`, if the Tenant **can** be updated during the HTTP request's lifetime, i.e. before the request times out. For example, a fast update could be achieved, if the Tenant update process requires only one or multiple updates of entries in a database.

- **b) Slow Update**

The *Application Provider* should respond early with `'202' Accepted`, if the Tenant **cannot** be updated during the HTTP request's lifetime, to avoid a timeout of the request. For example, a slow update has to be assumed, if the update of Tenant properties requires a long-running system update procedure, a downtime, or even manual intervention.

If unsure, a "Slow Update" should be assumed by the *Application Provider*.

A successful response will return the *Application Provider's* identifier of the (to-be) updated Tenant in the payload as well as in the `Location` header.

Hereby, the *Application Provider* has the option to return a different (temporary) resource identifier instead of the previously used resource identifier. This enables the *Application Provider* to return a response quickly, in case this action requires a (temporary) change of the resource identifier. For example, this identifier can point to an internal job resource that is performing the triggered action and used as a "proxy identifier" for progress tracking by the *Application Provider*.

Once / In case a new (permanent) identifier is available, the *Application Provider* can indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

From the perspective of the *Application Manager*, it can in all cases simply track the status of the resource by the provided `Location` header URI, such that the internal workings of the *Application Provider's* provisioning process are almost fully abstracted.

In case the update request was sent towards a moved Tenant resource identifier, the *Application Provider* must indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

AUTHORIZATIONS: > *Mutual TLS*

PATH PARAMETERS

`id` string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`

required

Examples: `b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24`

`888133312123288888`

The identifier of the Tenant resource.

REQUEST BODY SCHEMA: `application/json`

required

The updated Tenant specification with some / all mutable parts modified, when compared to the current state of the Tenant.

Immutable attributes can be provided as part of the request, but must be identical to the current state of the Tenant. Those attributes include `id`, `application`, `sapId`, `businessType`, `location`, and `status`.

The *Application Provider* should ignore the presence of immutable attributes, but can alternatively return `'409' Conflict`, if it can detect that a provided immutable value does not match the current state. This can support the *Application Manager* in detecting attribute state drifts early.

`customer` > object (Customer)
The individual customer.

`contract` > object (Contract)

	The contractual details that are associated with the Tenant.
<code>products ></code>	Array of Product ID only (object) or Product ID & Quota (object) (Products) <code>non-empty</code> The list of products that are associated to the Tenant.
<code>operationalType</code>	string (OperationalType) A specific operation performed on the Tenant by the <i>Application Provider</i>.
<code>initialUsers ></code>	Array of objects (InitialUsers) <code>non-empty</code> The initial users that should be granted immediate access to the Tenant.
<code>hostTenantSpecification ></code>	object (HostTenantSpecification) A wrapper for host tenant specific properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.
<code>mixinSpecifications ></code>	Array of objects (MixinSpecifications) <code>non-empty</code> A wrapper for generic properties that should be mixed into the regular Tenant specification. Includes <i>Application Provider</i> specific parameters that are not generically applicable and hence not well-defined in the SPFI protocol, but that are required for the Tenant provisioning in the context of a particular <i>Application Provider</i>.
<code>additionalProperties ></code>	object (AdditionalPropertiesFromManager) <code>[1 .. 100] properties</code> Freely definable properties, which are provided by the <i>Application Manager</i>. Those properties are not generally applicable, but only relevant for a specific <i>Application Provider</i> in order to successfully provision/manage a Tenant dependent on its specific configuration needs. Those properties will simply be routed through to the <i>Application Provider</i>, but won't be interpret or otherwise acted on by the <i>Application Manager</i>, because they are not well-defined on the SPFI protocol level.

Responses

> 200 OK.

Indicates that the Tenant was updated successfully and returns the *Application Provider's* identifier (`id`) of the Tenant.

The `Location` header can be checked for further details on the resource's status.

> 202 Accepted.

Indicates that the request did not immediately update the Tenant, but triggered an asynchronous update flow. Returns the *Application Provider's* (temporary) identifier (`id`) of the Tenant and the location of the Tenant's status in the `Location` header to track the update status.

Dependent on the offered capabilities of the *Application Provider* a `GET` request towards the individual Tenant resource can be used by the *Application Manager* in combination with the `Retry-After` header to get informed about future status updates of the Tenant resource in a timely manner. This allows to monitor the status of the update process. Therefore, the `Location` header should be checked, to observe the current status of the to-be updated resource.

> 400 Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

> 401 Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

> 403 Forbidden.

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

> 404 Not Found. The *Application Manager* provided credentials, but did request a resource that

- either does not exist
- or that exists, but the credentials are not sufficient to perform the desired action on the specified resource

In the latter case, the existence of the resource should be disguised from the *Application Manager* to provide additional security compared to a `'403' Forbidden`.

> 409 Conflict.

The *Application Manager* wanted to perform a resource update, but the resource is in a state that prohibits this action or the provided data updates an immutable attribute.

> 410 Gone.

The Tenant resource has been permanently moved by the *Application Provider* to the location specified by the `Location` header with a new resource identifier as specified in the response body.

The *Application Manager* should update its resource reference accordingly to the new location and use this location in subsequent requests, because the *Application Provider* can retire the redirect if it is certain that the new location is well known.

To enforce the explicit handling of this response, a `'410' Gone` is used in favor of a `'301' Moved Permanently`, because many HTTP clients follow `301` redirects transparently by default.

In that case, the *Application Manager* would handle only the response of the subsequent (forwarded) request explicitly, such that the resource relocation information within the `301` response would stay unnoticed.

> 429 Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

> 500 Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

PUT /tenants/{id}

Request samples

Payload**Content type**

application/json

Copy Expand all Collapse all

```
{
  "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24",
  - "application": {
    "globalTenantId": "12099f8d-73ce-4175-b53c-8080410d7ef8",
    + "endpoints": [ ... ],
    + "additionalProperties": { ... }
  },
  "sapId": "324928342811231213",
  - "customer": {
    "id": "324928342811231213",
    "erpId": "0001211321",
    "name": "string",
    + "contact": { ... }
  },
}
```

```
- "contract": {
  "validFrom": "2019-08-24",
  "validTo": "2019-08-24"
},
- "products": [
  + { ... }
],
"businessType": "ZH421",
"operationalType": "string",
- "location": {
  + "geography": { ... },
  + "infrastructure": { ... }
},
- "initialUsers": [
  + { ... }
],
- "hostTenantSpecification": {
  "group": "my-group",
  "version": "my-version",
  "type": "my-type",
  "spec": "{ \"p1\": \"v1\", \"p2\": \"v2\" }"
},
- "mixinSpecifications": [
  + { ... }
],
- "additionalProperties": {
  "managerCustomProperty": "manager-custom-value"
},
- "status": {
  "state": "in-activation",
  + "details": { ... },
  "lastModified": "2019-08-24T14:15:22Z"
}
}
```

Response samples

200	202	400	401	403	404	409	410	429
500	503							

Content type

application/json

Copy


```
{  
  "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24"  
}
```

Delete a Tenant

Delete a Tenant.

The *Application Provider* will check, if the Tenant is in a lifecycle state that allows its deletion - namely in state "active", "blocked", "in-recoverable-error", or "final-error".

If this check is not successful, the *Application Provider* should respond with a `'409' Conflict` indicating the conflicting state. If this check is successful and therefore the overall deletion request is accepted, the *Application Provider* should respond with a different status code dependent on the kind of deletion process that it supports:

- **a) Fast Deletion**

The *Application Provider* should respond with `'204' No content`, if the Tenant **can** be deleted during the HTTP request's lifetime, i.e. before the request times out.

- **b) Slow Deletion**

The *Application Provider* should respond early with `'202' Accepted`, if the Tenant **cannot** be deleted during the HTTP request's lifetime, to avoid a timeout of the request.

This indicates that the request was accepted for processing, but this processing is not completed yet or might not have started. Afterwards, the Tenant's deletion status can be monitored via `GET /tenants/{id}/status`.

If unsure, a "Slow Deletion" should be assumed by the *Application Provider*.

In case of a "Slow Deletion", a successful response will return the *Application Provider's* identifier of the to-be deleted Tenant in the payload as well as in the `Location` header.

Hereby, the *Application Provider* has the option to return a different resource identifier instead of the previously used resource identifier. This enables the *Application Provider* to return a response quickly, in case this action requires a change of the resource identifier. For example, this identifier can point to an internal job resource that is performing the triggered action and used as a "proxy identifier" for progress tracking by the *Application Provider*.

In case the deletion request was sent towards a moved Tenant resource identifier, the *Application Provider* must indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

Unlike other lifecycle state transitions, the deletion of a Tenant cannot be revoked.

AUTHORIZATIONS: > *Mutual TLS*

PATH PARAMETERS

id
required

string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`

Examples: `b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24`

`888133312123288888`

The identifier of the Tenant resource.

Responses

> 202 Accepted.

Indicates that the request did not immediately delete the Tenant, but triggered an asynchronous deletion flow. Returns the *Application Provider's* (temporary) identifier (`id`) of the Tenant and the location of the Tenant's status in the `Location` header to track the deletion status.

Dependent on the offered capabilities of the *Application Provider* a `GET` request towards the individual Tenant resource can be used by the *Application Manager* in combination with the `Retry-After` header to get informed about future status updates of the Tenant resource in a timely manner. This allows to monitor the status of the deletion. Therefore, the `Location` header should be checked, to observe the current status of the to-be deleted resource.

— 204 No Content. The request was successful and the resource has been removed.

> 400 Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

> 401 Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

> 403 Forbidden.

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

> 404 Not Found. The *Application Manager* provided credentials, but did request a resource that

- either does not exist
- or that exists, but the credentials are not sufficient to perform the desired action on the specified resource

In the latter case, the existence of the resource should be disguised from the *Application Manager* to provide additional security compared to a `'403' Forbidden`.

> 409 Conflict.

The *Application Manager* wanted to delete the resource, but the resource is in a state that prohibits this lifecycle state transition.

> 410 Gone.

The Tenant resource has been permanently moved by the *Application Provider* to the location specified by the `Location` header with a new resource identifier as specified in the response body.

The *Application Manager* should update its resource reference accordingly to the new location and use this location in subsequent requests, because the *Application Provider* can retire the redirect if it is certain that the new location is well known.

To enforce the explicit handling of this response, a `'410' Gone` is used in favor of a `'301' Moved Permanently`, because many HTTP clients follow `301` redirects transparently by default.

In that case, the *Application Manager* would handle only the response of the subsequent (forwarded) request explicitly, such that the resource relocation information within the `301` response would stay unnoticed.

> 429 Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

> 500 Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

DELETE `/tenants/{id}`

Response samples

400**401****403****404****409****410****429****500****503****Content type**

application/json

Example

Error: Invalid Request Body

[Copy](#)[Expand all](#)[Collapse all](#)

```

{
  - "error": {
    "code": "400-01",
    "message": "Invalid message body.",
    "displayMessage": "^. $",
    "target": "http://example.com",
    + "details": [ ... ]
  }
}

```

Get status of a Tenant

Get the status of a single Tenant.

This information includes the current provisioning status of the Tenant.

During initial provisioning or other actions performed on a Tenant, it can be beneficial for an *Application Manager* to get information about status changes timely. The *Application Manager* is advised to rely on the received `Retry-After` header on a `'200' OK` response (if available) to establish an optimal polling interval.

The *Application Provider* must only grant access to a Tenant's information, if it is associated to the querying *Application Manager* based on the confirmed identity of the *Manager* defined by Mutual TLS. In case the request was sent towards a moved Tenant resource identifier, the *Application Provider* must indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

AUTHORIZATIONS: > *Mutual TLS*

PATH PARAMETERS

`id` string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`

required

Examples: `b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24`

`888133312123288888`

The identifier of the Tenant resource.

Responses

> **200 OK**. Returns the current status information of the resource.

If any of the resources' status is non-final, a `Retry-After` header is sent with the response.

Hereby, the *Application Provider* tries to provide a best guess for the *Application Manager* on when to ask for an updated status.

➤ **400** Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

➤ **401** Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

➤ **403** Forbidden.

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

➤ **404** Not Found. The *Application Manager* provided credentials, but did request a resource that

- either does not exist
- or that exists, but the credentials are not sufficient to perform the desired action on the specified resource

In the latter case, the existence of the resource should be disguised from the *Application Manager* to provide additional security compared to a `'403' Forbidden`.

➤ **410** Gone.

The Tenant resource has been permanently moved by the *Application Provider* to a new resource identifier as specified in the response body, such that its status can be found at the location specified by the `Location` header.

The *Application Manager* should update its resource reference accordingly to the new location and use this location in subsequent requests, because the *Application Provider* can retire the redirect if it is certain that the new location is well known.

To enforce the explicit handling of this response, a `'410' Gone` is used in favor of a `'301' Moved Permanently`, because many HTTP clients follow `301` redirects transparently by default.

In that case, the *Application Manager* would handle only the response of the subsequent (forwarded) request explicitly, such that the resource relocation information within the `301` response would stay unnoticed.

➤ **429** Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

➤ **500** Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

➤ **503** Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

GET /tenants/{id}/status

Response samples

200

400

401

403

404

410

429

500

503

Content type

application/json

Copy

Expand all

Collapse all

```
{
  "state": "in-activation",
  - "details": {
    "code": "string",
    "message": "string",
    "displayMessage": "^. $"
  },
  "lastModified": "2019-08-24T14:15:22Z"
}
```

Change desired lifecycle state of a Tenant

Change the desired lifecycle state of a Tenant by either blocking, unblocking, or resurrecting it.

Dependent on the implementation at the *Application Provider* side, the individual state transition can be performed synchronously or asynchronously.

A successful response will return the *Application Provider's* identifier of the (to-be) transitioned Tenant in the payload as well as in the `Location` header.

Hereby, the *Application Provider* has the option to return a different (temporary) resource identifier instead of the previously used resource identifier. This enables the *Application Provider* to return a response quickly, in case this action requires a (temporary) change of the resource identifier. For example, this identifier can point to an internal job resource that is performing the triggered action and used as a "proxy identifier" for progress tracking by the *Application Provider*.

Once / In case a new (permanent) identifier is available, the *Application Provider* can indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

From the perspective of the *Application Manager*, it can in all cases simply track the status of the resource by the provided `Location` header URI, such that the internal workings of the *Application Provider's* provisioning process are almost fully abstracted.

The following lifecycle state transitions are possible:

Block

Blocks the use of a Tenant by setting its desired lifecycle state to `blocked`, transitioning it into a mode where the *Application Consumer* loses all access to the Tenant (write **and** read).

The use of a Tenant is blocked as the first step whenever a contract expires, which granted an *Application Consumer* the permission to use the Tenant. A contract can expire, if

- the contract duration expired without a renewal of the contract.
- the necessary reoccurring payments were not made during the lifetime of the contract.

Alternatively, a Tenant can be set to blocked explicitly by a triggered Tenant deactivation by the *Application Consumer*. This will in turn trigger the *Application Manager* to inform the *Application Provider* of the *Application Consumer's* decision by setting the desired lifecycle state to "blocked". Within this state, an *Application Consumer* can no longer access the Tenant or any information that it holds, nor backup the Tenant data. However, an *Application Consumer* has the possibility to unblock the Tenant for a certain while without losing the Tenant data / configuration.

If a contract is not renewed or the Tenant is not unblocked by other means within a defined time frame, it is expected that the Tenant gets eventually deleted entirely.

This action can only be performed when the Tenant is in the `active` state. Otherwise, a `'409' Conflict` must be returned.

Unblock

Unblocks a Tenant by setting its desired lifecycle state to `active`, making it read/write-able again. If successful, normal interaction with the Tenant can be resumed as before it was blocked.

This action can only be performed when the Tenant is in the `blocked` state. Otherwise, a `'409' Conflict` must be returned.

Resurrect

Resurrects a Tenant by actively setting its desired lifecycle state to `active` or `blocked` again.

This instructs the *Application Provider* to actively retry a previously failed lifecycle transition, or to try to recover into the desired lifecycle state from an error, or to avoid an unintentional deletion (if feasible).

This action can only be performed when the Tenant is in the `in-recoverable-error`. Otherwise, a `'409' Conflict` must be returned.

In case the request was sent towards a moved Tenant resource identifier, the *Application Provider* must indicate the move of the resource to the *Application Manager* by the use of a `'410' Gone` in subsequent requests towards this resource and inform the *Application Manager* about this new location accordingly.

AUTHORIZATIONS: > *Mutual TLS*

PATH PARAMETERS

`id` string (TenantResourceId) `^[a-zA-Z0-9-._~]*$`

required

Examples: `b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24`

`888133312123288888`

The identifier of the Tenant resource.

REQUEST BODY SCHEMA: `application/json`

required

The state that the Tenant should be in.

`state` >

Active (string) or Blocked (string) (LifecycleStatesDesiredActionableWithBlocked)

Responses

> 200 OK.

Indicates that the Tenant was synchronously updated to the desired lifecycle state and returns the *Application Provider's* identifier (`id`) of the Tenant.

The `Location` header can be checked for further details on the Tenant's status.

> 202 Accepted.

Indicates that the request did not immediately update the Tenant's lifecycle status, but triggered an asynchronous state transitioning flow. Returns the *Application Provider's* (temporary) identifier (`id`) of the Tenant and the location of the Tenant's status in the `Location` header to track the lifecycle state transitioning status.

Dependent on the offered capabilities of the *Application Provider* a `GET` request towards the individual Tenant resource status can be used by the *Application Manager* in combination with the `Retry-After` header to get informed about future status updates of the Tenant resource in a timely manner.

This allows to monitor the status of the lifecycle state transition. Therefore, the `Location` header should be checked, to observe the current status of the to-be transitioned resource.

> 400 Bad Request.

The *Application Manager* used unknown or invalid values in the request's message body and/or URL query parameters.

> 401 Unauthorized.

The *Application Manager* did not provide credentials or the credentials were invalid.

> 403 Forbidden.

The *Application Manager* did provide credentials, but the credentials are not sufficient to perform the desired action on the specified resource.

> 404 Not Found. The *Application Manager* provided credentials, but did request a resource that

- either does not exist
- or that exists, but the credentials are not sufficient to perform the desired action on the specified resource

In the latter case, the existence of the resource should be disguised from the *Application Manager* to provide additional security compared to a `'403' Forbidden`.

> 409 Conflict.

The *Application Manager* wanted to perform a state transition for a resource, but the resource is in a state that prohibits this transition.

> 410 Gone.

The Tenant resource has been permanently moved by the *Application Provider* to a new resource identifier as specified in the response body, such that its status can be found at the location specified by the `Location` header.

The *Application Manager* should update its resource reference accordingly to the new location and use this location in subsequent requests, because the *Application Provider* can retire the redirect if it is certain that the new location is well known.

To enforce the explicit handling of this response, a `'410' Gone` is used in favor of a `'301' Moved Permanently`, because many HTTP clients follow `301` redirects transparently by default.

In that case, the *Application Manager* would handle only the response of the subsequent (forwarded) request explicitly, such that the resource relocation information within the `301` response would stay unnoticed.

> 429 Too Many requests.

The *Application Manager's* request is potentially valid, but the *Application Provider* is unwilling to handle the request due to too many consecutive requests in a short period of time by the *Application Manager*. This can be used for rate limiting purposes.

The *Application Provider* is expected to return a `Retry-After` header, which indicates the time after which it will be again willing to accept requests from the *Application Manager*.

> 500 Internal Server Error.

The *Application Manager's* request is potentially valid, but the *Application Provider* encountered an unexpected condition that prevented it from fulfilling the request.

> 503 Service Unavailable.

The *Application Manager's* request is potentially valid, but the *Application Provider* is currently unable to handle the request.

This is expected to be a temporary issue, like a maintenance downtime. If possible, the estimated recovery-time should be specified in the `Retry-After` header, such that the *Application Manager* will retry its request timely after this window.

POST `/tenants/{id}/status`

Request samples

Payload

Content type

application/json

Copy

```
{  
  "state": "active"  
}
```

Response samples

200	202	400	401	403	404	409	410	429
500	503							

Content type
application/json

Copy

```
{
  "id": "b8d730dc-2e8f-4ddd-b6ea-190fdffc2c24"
}
```